



MINISTERIO DEL PODER POPULAR PARA LA EDUCACIÓN SUPERIOR

UNIVERSIDAD SIMÓN BOLÍVAR

COORDINACIÓN DE CICLO BÁSICO

SARTENEJAS, DISTRITO CAPITAL.

Informe

Etapas 2 – Analizador sintáctico

Estudiantes:

Alejandro Villamizar 22-10439

Francisbell Graterol 22-10177

Carlos Bisogno 22-10041

Sartenejas, junio 2025.

1. Propósito

Este informe describe la formulación e implementación del analizador sintáctico del lenguaje BOT, construido con la herramienta PLY (módulo yacc) sobre los tokens producidos por el analizador léxico. El analizador recibe la secuencia de tokens y construye un árbol de sintaxis abstracta (AST) que representa la estructura del programa fuente, validando que dicha secuencia sea derivable según la gramática definida para BOT.

2. Estructura general de la implementación

El archivo parser.py se organiza en tres partes: la definición de los nodos del AST, las reglas de la gramática (funciones `p_*`) y la rutina de impresión del árbol (`imprimir_arbol`).

Nodos del AST

Cada construcción sintáctica de BOT tiene asociada una clase de nodo (por ejemplo, `IfNode`, `WhileNode`, `StoreNode`, `ContNodes`). Cada clase almacena únicamente los datos necesarios para representar esa construcción: una condición y un cuerpo en el caso de `IfNode` y `WhileNode`, una acción y una lista de variables en `ContNodes`, etc. Las expresiones aritméticas, booleanas y relacionales comparten una forma binaria (operando izquierdo, operador, operando derecho), por lo que se modelan con tres clases paralelas — `AritOpNode`, `BoolOpNode` y `RelOpNode` — en lugar de una única clase genérica, de manera que cada una pueda fijar su propio atributo `type` (int o bool) sin necesidad de inferirlo después.

Reglas de la gramática

Las reglas siguen la estructura del enunciado: un programa (programa) se compone de una sección opcional de declaraciones (`bot_list`) y un controlador principal (`statement_list`). Cada declaración de robot (`bot_def`) agrupa un tipo, una lista de identificadores y, opcionalmente, una lista de comportamientos (`on_list`). Dentro del controlador y de los comportamientos se reutiliza la misma regla `statement_list`, lo que permite que las instrucciones se compongan por secuenciación sin distinguir si se ejecutan en el controlador o dentro de un robot.

Las expresiones se definen mediante una única regla no terminal (`expression`) con producciones para cada operador. La precedencia y asociatividad de los operadores se controla con la tabla `precedence` nativa de PLY, que ordena los operadores de menor a mayor precedencia (disyunción, conjunción, relacionales, suma/resta, multiplicación/división/módulo,

negación) y resuelve los conflictos de desplazamiento/reducción que produce una gramática ambigua. El operador unario de negación aritmética (símbolo UMENOS) se declara aparte con mayor precedencia que la resta binaria, para que $-3 + 2$ se interprete como $(-3) + 2$ y no como $-(3 + 2)$.

Las instrucciones de robot que comparten una forma similar (almacenamiento y soltado, por ejemplo) se mantienen como reglas separadas aun cuando su estructura sintáctica sea parecida, porque cada una construye un nodo distinto y tiene una semántica distinta en etapas posteriores del proyecto. En contraste, las instrucciones de entrada que comparten exactamente la misma forma — lectura desde la entrada estándar y colección desde la matriz, ambas con la forma instrucción [as identificador] — se mantienen también como reglas y nodos separados (ReadNode y CollectNode), ya que, aunque su sintaxis es idéntica, su origen de datos y su validación en el análisis de contexto son distintos.

Manejo de errores sintácticos

Ante un error sintáctico, la función `p_error` reporta el token inesperado junto con su línea y detiene la ejecución de inmediato (`sys.exit(1)`). Se aborta en el primer error en lugar de acumular varios, a diferencia del analizador léxico, que sí acumula errores antes de reportarlos.

3. Construcción y representación del AST

Las listas (lista de bots, lista de identificadores, lista de comportamientos, lista de instrucciones) se construyen con un patrón recursivo por la izquierda: la primera producción agrega un elemento a una lista ya existente y la segunda crea la lista a partir de un único elemento. Las clases correspondientes (BotListNode, VarListNode, EstListNodes) exponen un método `append` para no reconstruir la lista en cada paso de la reducción.

El árbol resultante se imprime mediante la función `imprimir_arbol`, que recorre el AST de forma recursiva según el tipo de cada nodo (`isinstance`) y produce una representación textual indentada. Esta función solo recorre la sección del controlador principal (`execute`); la sección de declaraciones (`create`) no se incluye en la impresión del árbol.

4. Ajustes realizados sobre la versión inicial

Durante la revisión de la primera versión del analizador se identificaron varias instrucciones que no estaban completas frente al enunciado. A continuación, se resume cada ajuste.

- **Lectura (read):** no existía como instrucción independiente. Se agregó la regla `p_statement_read`, con la forma `read [as identificador]`, y el nodo `ReadNode` con un atributo `var_name` que es `None` cuando no está presente la cláusula `as`.
- **Colección (collect as):** la regla original solo aceptaba `collect` sin argumentos. Se separó en su propia regla `p_statement_collect` con la forma `collect [as identificador]`, análoga a `read`, usando el nodo `CollectNode`.
- **Soltado (drop):** la regla original trataba `drop` como instrucción sin argumentos. Según el enunciado, `drop` siempre recibe una expresión obligatoria (`drop (Expresión)`), por lo que se reemplazó por `p_statement_drop`, con la forma `drop expression`, y el nodo `DropNode`.
- **Movimiento (left/right/up/down):** estas cuatro instrucciones solo se aceptaban sin argumentos. El enunciado indica que la expresión de unidades a avanzar es opcional. Se reemplazaron por `p_statement_movimiento`, con cuatro formas con expresión y cuatro sin ella, unificadas en el nodo `MovimientoNode` (atributos `direccion` y `expr`, este último `None` si no se indicó cantidad).
- **Literales de carácter:** se agregó la clase `CharNode` para representar los literales `char` con un nombre propio en el AST, en lugar de un `Node` genérico. Esta clase resuelve las secuencias de escape `\n`, `\t` y `\'` descritas en el enunciado, dejando en `value` el carácter real que representan.
- **Formato de impresión del AST:** se rediseñó `imprimir_arbol` para seguir un formato de indentación uniforme de cuatro espacios por nivel, con etiquetas de la forma "`- etiqueta: valor`" para los datos de cada nodo, y se agregaron los casos faltantes que no estaban contemplados en la versión original.